

AULA 13

Revisão



Prof.: Allan Patrick de Freitas Santana
Curso: Ciência da Computação · UNIP
Duração sugerida: 3h a 4h

BLOCO 1

Gerência de memória e processamento

Gerência de memória: ideia central

O SO precisa entregar a ilusão de memória simples, protegida e suficiente.

O que o sistema operacional controla

- Quais regiões de memória pertencem a cada processo.
- Como impedir que um processo invada a memória de outro.
- Como mapear endereços lógicos para endereços físicos.
- Como usar disco como apoio quando a RAM não é suficiente.

Palavra-chave

Abstração: o programa vê memória lógica; o hardware executa em memória física.

Risco clássico

Sem isolamento, um bug ou programa malicioso poderia sobrescrever dados de outro processo.

Conexão com Linux

Comandos como free, top, ps e vmstat ajudam a observar uso real.

Memória virtual e paginação

A memória virtual separa endereço usado pelo programa do endereço físico real.

Conceitos essenciais

- Página: bloco de tamanho fixo no espaço lógico do processo.
- Quadro/frame: bloco de mesmo tamanho na memória física.
- Tabela de páginas: estrutura que traduz página lógica para frame físico.
- Page fault: ocorre quando a página necessária não está carregada na RAM.

endereço lógico

↓

[página | deslocamento]

↓

tabela de páginas

[frame | deslocamento]

O deslocamento não muda na tradução; muda a página para o frame correspondente.

Exercício 1 — paginação

Calcule página, deslocamento e endereço físico.

Enunciado

Um sistema usa páginas de 1 KiB. Um processo acessa o endereço lógico 2500. A tabela informa: página 0 → frame 5, página 1 → frame 2, página 2 → frame 8. Qual é o endereço físico?

Passo a passo esperado

- 1 KiB = 1024 bytes.
- Página = endereço lógico // tamanho da página.
- Deslocamento = endereço lógico % tamanho da página.
- Endereço físico = frame × tamanho da página + deslocamento.

```
2500 // 1024 = 2
2500 % 1024 = 452
página 2 → frame 8
8×1024 + 452 = 8644
```

Gabarito

Endereço físico = 8644.

Swap, page fault e thrashing

Nem todo uso de swap é problema; o problema é depender dele continuamente.

Como interpretar

- Swap permite mover páginas menos usadas para o disco.
- Page fault ocorre quando a página pedida não está presente na RAM.
- Thrashing acontece quando o sistema passa mais tempo trocando páginas do que executando trabalho útil.
- Sintoma comum: disco muito ativo, CPU pouco produtiva e sistema lento.

Não confundir

Ter swap configurado é normal. Viver de swap é problema de desempenho.

Pergunta de diagnóstico

A lentidão vem de CPU ocupada ou de falta de memória causando page faults?

```
free -h  
vmstat 1  
top
```

Gerência de processamento

O processador é compartilhado; o escalonador decide quem executa e quando.

O que revisar

- Processo pronto não está necessariamente executando.
- Processo bloqueado aguarda evento, I/O ou recurso.
- Escalonador escolhe entre processos prontos.
- Preempção permite retirar a CPU de um processo para dar a outro.

CPU-bound

Processo que usa muita CPU e faz pouca E/S.

I/O-bound

Processo que alterna CPU curta com espera por entrada/saída.

Conexão com escalonamento

Políticas diferentes favorecem resposta, throughput ou justiça.

Métricas de escalonamento

Métrica	Fórmula	Interpretação
Turnaround	fim - chegada	tempo total do processo no sistema
Waiting	turnaround - burst	tempo esperando na fila de pronto
Response	1ª execução - chegada	tempo até receber CPU pela 1ª vez
Throughput	processos concluídos / tempo	vazão do sistema

Dica

Sempre monte uma linha do tempo. Sem linha do tempo, a chance de errar waiting e response aumenta muito.

Exercício 2 — FCFS de CPU

Calcule tempo de conclusão, turnaround e waiting.

Enunciado

Três processos chegam ao sistema: P1 chegada 0 burst 6; P2 chegada 2 burst 4; P3 chegada 4 burst 2. Usando FCFS, calcule conclusão, turnaround e waiting.

Proc	Chegada	Burst	Início	Fim	Turn.	Waiting
P1	0	6	0	6	$6-0=6$	$6-6=0$
P2	2	4	6	10	$10-2=8$	$8-4=4$
P3	4	2	10	12	$12-4=8$	$8-2=6$

Gabarito

Waiting médio = $(0 + 4 + 6) / 3 = 3,33$. Turnaround médio = $(6 + 8 + 8) / 3 = 7,33$.

BLOCO 2

Permissões de acesso e execução

Permissões em arquivos e diretórios

A leitura de rwx muda quando o alvo é um arquivo ou um diretório.

Permissão	Em arquivo	Em diretório
r	ler conteúdo	listar nomes
w	alterar conteúdo	criar/remover/renomear entradas
x	executar programa/script	atravessar/acessar caminho

Pegadinha clássica

Para acessar /home/aluno/docs/a.txt, não basta ter permissão no arquivo: é preciso ter x nos diretórios intermediários do caminho.

chmod, chown, chgrp e umask

Os comandos alteram permissões, dono, grupo e padrão de criação.

```
chmod 755 app.sh
chmod u+x script.sh
chown allan arquivo.txt
chgrp alunos arquivo.txt
umask 022
```

Como explicar

- 755 = dono rwx, grupo r-x, outros r-x.
- u+x adiciona execução ao usuário dono.
- chown troca o proprietário.
- chgrp troca o grupo.
- umask remove permissões padrão no momento da criação.

Dica de prova

Permissão final costuma ser: permissão base – umask. Arquivo normalmente parte de 666; diretório parte de 777.

Bits especiais: setuid, setgid e sticky bit

Esses bits aparecem em administração real.

setuid

Executável roda com permissões do dono do arquivo. Ex.: passwd precisa alterar dados protegidos.

setgid

Arquivo executa com grupo do arquivo ou diretório força herança do grupo.

sticky bit

Em diretórios compartilhados, só dono do arquivo, dono do diretório ou root remove. Ex.: /tmp.

```
chmod 4755 programa    # setuid
chmod 2755 pasta       # setgid
chmod 1777 /tmp        # sticky bit
```

Exercício 3 — permissões

Leia a permissão e explique o que cada perfil pode fazer.

Enunciado

Um arquivo possui permissão -rwxr-x---, dono allan e grupo alunos. O que o dono, o grupo e outros usuários podem fazer?

Resolução

- Dono: rwx → pode ler, escrever e executar.
- Grupo: r-x → pode ler e executar, mas não escrever.
- Outros: --- → não têm acesso.
- O primeiro caractere “-” indica arquivo regular.

Gabarito

Dono: leitura/escrita/execução. Grupo: leitura/execução.
Outros: nenhum acesso.

Por quê?

A string é lida em blocos: tipo + dono + grupo + outros.

BLOCO 3

Deadlock matemático: algoritmo do banqueiro

Como calcular Need, sequência segura e concessão de requisição.

Algoritmo do banqueiro: roteiro de resolução

O objetivo é verificar se o sistema fica em estado seguro.

Procedimento

- Calcular $\text{Need} = \text{Max} - \text{Allocation}$.
- Comparar Need de cada processo com Available.
- Se $\text{Need} \leq \text{Available}$, o processo pode terminar.
- Ao terminar, ele devolve sua Allocation para Available.
- Repetir até todos terminarem ou até travar.

Estado seguro

Existe pelo menos uma ordem em que todos os processos conseguem terminar.

Estado inseguro

Não prova deadlock imediato, mas indica risco: não há garantia de conclusão.

Exercício 4 – banqueiro

Verifique se o estado é seguro.

Enunciado

Recursos A, B, C. Available = (3,3,2). Calcule Need e encontre uma sequência segura, se existir.

Proc	Allocation	Max	Need	Obs.
P0	0 1 0	7 5 3	?	
P1	2 0 0	3 2 2	?	
P2	3 0 2	9 0 2	?	
P3	2 1 1	2 2 2	?	
P4	0 0 2	4 3 3	?	

Tarefa

Calcular Need = Max – Allocation antes de tentar a sequência.

Resolução 4.1 — cálculo do Need

Primeiro calcule o que cada processo ainda precisa para terminar.

Proc	Allocation	Max	Need	Como calcular
P0	0 1 0	7 5 3	7 4 3	(7-0, 5-1, 3-0)
P1	2 0 0	3 2 2	1 2 2	(3-2, 2-0, 2-0)
P2	3 0 2	9 0 2	6 0 0	(9-3, 0-0, 2-2)
P3	2 1 1	2 2 2	0 1 1	(2-2, 2-1, 2-1)
P4	0 0 2	4 3 3	4 3 1	(4-0, 3-0, 3-2)

Primeira comparação

Available inicial = (3,3,2). P1 e P3 já conseguem terminar, pois seus Needs cabem no Available.

Resolução 4.2 — sequência segura

A cada processo finalizado, os recursos alocados voltam para Available.

Passo	Escolha	Available antes	Devolve Allocation	Available depois
1	P1	3 3 2	2 0 0	5 3 2
2	P3	5 3 2	2 1 1	7 4 3
3	P4	7 4 3	0 0 2	7 4 5
4	P0	7 4 5	0 1 0	7 5 5
5	P2	7 5 5	3 0 2	10 5 7

Gabarito

O estado é seguro. Uma sequência segura possível é: P1 → P3 → P4 → P0 → P2.

Exercício 5 — requisição no banqueiro

Teste se uma nova requisição pode ser concedida.

Enunciado

No mesmo cenário, P1 solicita Request = (1,0,2). O sistema pode conceder imediatamente?

Critérios

- Request $\leq$ Need do processo?
- Request $\leq$ Available atual?
- Após conceder, o estado continua seguro?
- Se sim, pode conceder. Se não, deve esperar.

```
Need(P1) = (1,2,2)
Request = (1,0,2)
Available = (3,3,2)
```

Resposta

Passa nos dois testes iniciais e ainda há sequência segura. Pode conceder.

BLOCO 4

Escalonamento de disco

Cálculo de deslocamento total da cabeça do disco.

Como resolver questões de disco

A conta principal é somar deslocamentos absolutos entre cilindros atendidos.

Roteiro

- Identificar posição inicial da cabeça.
- Listar a fila de requisições.
- Aplicar a política pedida: FCFS, SSTF, SCAN ou C-SCAN.
- Somar $|\text{posição atual} - \text{próxima posição}|$.
- Comparar total de deslocamento.

FCFS

Atende na ordem de chegada.

SSTF

Atende primeiro a requisição mais próxima.

SCAN / C-SCAN

Varrem em direção definida, como elevador.

Exercício 6 — fila de disco

Calcule FCFS, SSTF, SCAN e C-SCAN.

Enunciado

Disco com cilindros 0 a 199. Cabeça inicial no cilindro 50. Fila: 82, 170, 43, 140, 24, 16, 190. Para SCAN e C-SCAN, considere direção inicial crescente.

O que entregar

- Ordem de atendimento em cada política.
- Deslocamento total da cabeça.
- Qual política foi melhor nesse cenário.

Observação didática

A melhor política pode variar conforme fila, posição inicial e direção. Não decore: simule.

Resolução 6.1 — FCFS

FCFS não otimiza deslocamento; apenas respeita a ordem original.

50 → 82 → 170 → 43 → 140 → 24 → 16 → 190

Conta

- $|50-82|=32$; $|82-170|=88$; $|170-43|=127$; $|43-140|=97$;
- $|140-24|=116$; $|24-16|=8$; $|16-190|=174$.
- Total = $32 + 88 + 127 + 97 + 116 + 8 + 174 = 642$ cilindros.

Gabarito FCFS

Deslocamento total = 642.

Resolução 6.2 — SSTF

SSTF sempre escolhe a requisição mais próxima da posição atual.

50 → 43 → 24 → 16 → 82 → 140 → 170 → 190

Conta

- $|50-43|=7$; $|43-24|=19$; $|24-16|=8$; $|16-82|=66$;
- $|82-140|=58$; $|140-170|=30$; $|170-190|=20$.
- Total = $7 + 19 + 8 + 66 + 58 + 30 + 20 = 208$ cilindros.

Gabarito SSTF

Deslocamento total = 208.

Resolução 6.3 — SCAN

SCAN segue uma direção até o fim e depois retorna.

50 → 82 → 140 → 170 → 190 → 199 → 43 → 24 → 16

Conta

- $|50-82|=32$; $|82-140|=58$; $|140-170|=30$; $|170-190|=20$; $|190-199|=9$;
- $|199-43|=156$; $|43-24|=19$; $|24-16|=8$.
- Total = $32 + 58 + 30 + 20 + 9 + 156 + 19 + 8 = 332$ cilindros.

Gabarito SCAN

Deslocamento total = 332, considerando ida até o extremo 199.

Resolução 6.4 — C-SCAN

C-SCAN atende em uma direção; ao chegar ao fim, volta ao início sem atender no retorno.

50 → 82 → 140 → 170 → 190 → 199 → 0 → 16 → 24 → 43

Conta

- 50 até 199: 149 cilindros. Retorno 199 até 0: 199 cilindros.
- Depois 0 → 16 → 24 → 43: $16 + 8 + 19 = 43$ cilindros.
- Total = $149 + 199 + 43 = 391$ cilindros.

Gabarito C-SCAN

Deslocamento total = 391, contando o retorno de 199 para 0.

Comparação das políticas

A política com menor deslocamento no exemplo não é necessariamente sempre a melhor.

Política	Ordem	Deslocamento	Comentário
FCFS	ordem original	642	simples, mas ruim aqui
SSTF	mais próximo	208	menor deslocamento aqui
SCAN	elevador	332	mais justo que SSTF em muitos cenários
C-SCAN	circular	391	tempo de espera mais uniforme

Fechamento

A resposta correta precisa mostrar a ordem e a soma; só citar o algoritmo não resolve a questão.

BLOCO 5

Blocos, fragmentação e RAID

Contas rápidas para sistema de arquivos e armazenamento.

Fragmentação e tamanho de bloco

O tamanho do bloco influencia desperdício e desempenho.

Ideia central

- Arquivo ocupa uma quantidade inteira de blocos.
- Se o último bloco não for preenchido, há fragmentação interna.
- Blocos maiores reduzem metadados, mas podem desperdiçar espaço em arquivos pequenos.
- Fragmentação externa aparece quando o espaço livre fica espalhado.

Fórmula útil

$\text{blocos necessários} = \text{teto}(\text{tamanho do arquivo} / \text{tamanho do bloco})$

Desperdício interno

$\text{blocos alocados} \times \text{bloco} - \text{tamanho real do arquivo}$

Exercício 7 — blocos e desperdício

Calcule quantidade de blocos e fragmentação interna.

Enunciado

Um sistema usa blocos de 4 KiB. Um arquivo possui 10 KiB. Quantos blocos são alocados? Quanto espaço sobra no último bloco?

```
blocos = teto(10 / 4)
blocos = teto(2,5) = 3
espaço alocado = 3×4 KiB = 12 KiB
```

Conclusão

- Arquivo real: 10 KiB.
- Espaço reservado: 12 KiB.
- Sobra interna: 2 KiB.

Gabarito

São necessários 3 blocos. A fragmentação interna é 2 KiB.

Exercício 8 — muitos arquivos pequenos

Mostre por que blocos grandes podem desperdiçar espaço.

Enunciado

Um diretório possui 100 arquivos de 1 KiB. O sistema usa blocos de 8 KiB. Quanto espaço é reservado e quanto é desperdiçado?

```
cada arquivo ocupa 1 bloco  
100 arquivos × 8 KiB = 800 KiB  
dados reais: 100 × 1 KiB = 100 KiB  
desperdício = 700 KiB
```

Gabarito

Reservado: 800 KiB. Desperdiçado: 700 KiB.

Por quê?

Como a unidade mínima é 8 KiB, cada arquivo pequeno deixa 7 KiB vazios dentro do bloco.

RAID: o que comparar

RAID combina discos para ganhar desempenho, redundância ou ambos.

RAID	Ideia	Capacidade útil	Tolerância
0	striping sem redundância	$N \times \text{disco}$	0 discos
1	espelhamento	50% em par	1 disco por par
5	paridade distribuída	$(N-1) \times \text{disco}$	1 disco
10	espelho + striping	50%	1 por espelho

Dica

RAID não substitui backup. RAID ajuda disponibilidade; backup ajuda recuperação histórica.

Exercício 9 — capacidade RAID

Calcule capacidade útil e tolerância a falha.

Enunciado

Um servidor possui 4 discos de 2 TB. Calcule a capacidade útil em RAID 0, RAID 1, RAID 5 e RAID 10.

Nível	Conta	Capacidade útil	Tolerância
RAID 0	$4 \times 2 \text{ TB}$	8 TB	nenhuma
RAID 1	espelho 50%	4 TB	depende dos pares
RAID 5	$(4-1) \times 2 \text{ TB}$	6 TB	1 disco
RAID 10	50% de 8 TB	4 TB	1 por par

Gabarito

RAID 0: 8 TB; RAID 1: 4 TB; RAID 5: 6 TB; RAID 10: 4 TB.

BLOCO 6

Integração e simulado final

Checklist

Teoria

- Deadlock: condições, prevenção, evitação, detecção e recuperação.
- Sistema de arquivos: interface, implementação, metadados, alocação e consistência.
- Armazenamento: HDD, SSD, RAID, swap, cache e escalonamento.
- Memória/processamento: paginação, swap, métricas e CPU/I/O-bound.

Prática Linux

- Pacotes: dpkg, apt-get, apt-cache e repositórios.
- Serviços: systemctl e journalctl.
- Cron: agendamento e validação por log.
- Mídias: lsblk, blkid, mkfs, mount e fstab.
- NFS: exports, showmount, mount -t nfs.
- Permissões: chmod, chown, umask e bits especiais.

Laboratório

Cenário

Um servidor de laboratório está lento, precisa compartilhar arquivos por NFS, registrar tarefa periódica e corrigir permissões de uma pasta de entrega.

Missões dos alunos

- Verificar consumo de memória/processos com free, ps, top ou vmstat.
- Criar pasta /srv/entregas com grupo alunos e permissões adequadas.
- Configurar cron para registrar heartbeat em log.
- Verificar serviço NFS com systemctl e journalctl.
- Descrever como persistiria uma montagem em /etc/fstab.
- Entregar relatório explicando comandos e evidências.

Questão 1

Deadlock e banqueiro.

Questão

Em um sistema com recursos A e B, um processo P0 possui Allocation=(1,0), Max=(3,2) e Available=(2,1). O Need de P0 cabe no Available? P0 pode terminar neste momento?

Alternativas

- A) Need=(2,2), não cabe.
- B) Need=(2,2), cabe.
- C) Need=(3,2), cabe.
- D) Need=(1,1), não cabe.

Resolução

Need = Max - Allocation = (3-1, 2-0) = (2,2). Available=(2,1), então B falta.

Gabarito

Alternativa A.

Questão 2

Permissões e diretórios.

Questão

Um diretório possui permissão `drwxrwx--t`. O que o sticky bit indica nesse contexto?

Alternativas

- A) Qualquer usuário pode apagar qualquer arquivo.
- B) Apenas o dono do arquivo, dono do diretório ou root pode remover arquivos.
- C) O diretório será executado como root.
- D) Todos os arquivos herdarão o dono do diretório.

Resolução

Sticky bit em diretórios compartilhados protege arquivos contra remoção por outros usuários.

Gabarito

Alternativa B.

Questão 3

Disco e fragmentação.

Questão

Um arquivo de 17 KiB é armazenado em blocos de 4 KiB. Quantos blocos são usados e qual é o desperdício interno?

```
blocos = teto(17/4) = 5  
espaço alocado = 5×4 = 20 KiB  
desperdício = 20 - 17 = 3 KiB
```

Gabarito

5 blocos e 3 KiB de fragmentação interna.

Por quê?

O último bloco fica parcialmente vazio, mas continua reservado ao arquivo.

Gabarito

Ex.	Tema	Resposta-chave
1	Paginação	Endereço físico = 8644
2	FCFS CPU	Waiting médio 3,33; turnaround médio 7,33
3	Permissões	Dono rwx; grupo r-x; outros ---
4	Banqueiro	Seguro: P1→P3→P4→P0→P2
5	Request	Pode conceder
6	Disco	FCFS 642; SSTF 208; SCAN 332; C-SCAN 391
7	Blocos	3 blocos; 2 KiB desperdiçados
8	Arquivos pequenos	800 KiB reservado; 700 KiB desperdiçado
9	RAID	0=8TB; 1=4TB; 5=6TB; 10=4TB